ForecastLabAI — time-series backtesting and metrics

Problem

Random K-fold with shuffling mixes past and future. Explain how models are evaluated and chosen instead.

Instant solution

Splits (`backtesting/splitter.py`)

- **Expanding window:** training history grows fold by fold, validation moves forward.
- **Sliding window:** fixed-size training window slides forward.
- **Gap:** configurable gap between train end and validation start.

Metric	Measures
MAE	mean absolute error, in demand units
RMSE	punishes large errors more
WAPE	total abs error / total actual demand
sMAPE	scale-adjusted percentage error
Bias	systematic over- or under-forecast
Stability index	how consistent a model is across folds

All in `backtesting/metrics.py`, plus per-bucket metrics.

Selection (`model_selection/ranking.py`): deterministic ranking with a tie-break chain and a recommendation confidence based on relative improvement over alternatives.

Interview answer

> *I use walk-forward backtesting, not random cross-validation. Each model is evaluated on several chronological windows and compared to simple baselines. I look at bias and stability, not one metric, which gives a more realistic picture of behavior on unseen future periods.*

Why it works / gotchas

- Low backtest error ≠ production-safe: future availability of exogenous inputs, demand shifts, data quality and operations still matter.
- A model must beat seasonal naive on representative folds to justify its complexity.

Diagram brief

- Type: fold chart (Gantt-like), two panels.
- Panel "Expanding": Fold 1–3, training bar grows, validation bar moves right; optional gap block between them.
- Panel "Sliding": same, training bar keeps fixed length.
- Legend: Training · Gap · Validation · Later (unused). Note "illustrative, not measured results".

- Below: metrics → ranking (tie-break) → champion + confidence.